

Python Web Scraping

A Guide to Handling Different Data Formats

Covers: HTML, JSON, CSV, XML, APIs

Introduction

Web scraping is the process of automatically extracting data from websites and online sources. Python has a rich ecosystem of libraries that make it easy to handle different data formats — from raw HTML pages to structured JSON APIs and CSV files. This tutorial walks through each format with practical examples you can run immediately.

Required Libraries

Install all dependencies with:

```
pip install requests beautifulsoup4 lxml pandas
```

1. Scraping HTML Pages

HTML is the most common format you will encounter when scraping websites. The **requests** library fetches the page and **BeautifulSoup** parses the HTML structure so you can extract specific elements like headings, tables, or links.

Key Libraries

`requests` — sends HTTP GET/POST requests to fetch page content.

`beautifulsoup4` — parses HTML and lets you navigate the DOM tree.

Example: Extracting a Page Title and All Links

```
import requests from bs4 import BeautifulSoup url = 'https://example.com' response = requests.get(url) soup = BeautifulSoup(response.text, 'html.parser') # Get the page title title = soup.find('title').text print('Page Title:', title) # Get all hyperlinks links = soup.find_all('a') for link in links: print(link.get('href'))
```

Common BeautifulSoup Methods

`soup.find('tag')` — finds the first matching HTML tag.

`soup.find_all('tag')` — finds all matching tags, returns a list.

`soup.find('tag', {'class': 'name'})` — finds by tag and CSS class.

`element.text` — extracts the visible text inside a tag.

`element.get('href')` — extracts an attribute value from a tag.

2. Scraping JSON Data

JSON (JavaScript Object Notation) is the standard format for web APIs. When a URL returns JSON instead of HTML, you don't need BeautifulSoup — Python's built-in `json` module or the `.json()` method on a requests response handles it directly.

Example: Fetching Weather Data from wttr.in

```
import requests url = 'https://wttr.in/Johannesburg?format=j1' response =
requests.get(url) data = response.json() # converts JSON response to a Python dict #
Navigate the nested structure temp = data['current_condition'][0]['temp_C'] humidity
= data['current_condition'][0]['humidity'] feels_like =
data['current_condition'][0]['FeelsLikeC'] print(f'Temperature: {temp} C')
print(f'Humidity: {humidity}%') print(f'Feels Like: {feels_like} C')
```

Understanding JSON Structure

JSON data maps directly to Python types: objects become dictionaries, arrays become lists, strings stay strings, and numbers stay numbers. Use square brackets to navigate nested data.

`data['key']` — accesses a value by key in a dictionary.

`data['key'][0]` — accesses the first item in a list.

`data['key'][0]['nested_key']` — navigates deeper into nested structure.

Saving JSON Data to a File

```
import json # Save to file with open('weather_data.json', 'w') as f: json.dump(data,
f, indent=4) # Load from file with open('weather_data.json', 'r') as f: loaded_data =
json.load(f)
```

3. Handling CSV Data

CSV (Comma-Separated Values) files are commonly used for tabular data exports. Python's **pandas** library is the most efficient way to read, clean, and manipulate CSV data.

Example: Reading and Cleaning a CSV File

```
import pandas as pd # Read CSV from a URL or local file df =  
pd.read_csv('https://example.com/data.csv') # View first 5 rows print(df.head()) #  
Remove rows with null/empty values df = df.dropna() # Rename a column df =  
df.rename(columns={'old_name': 'new_name'}) # Filter rows where temperature > 20 df =  
df[df['temperature'] > 20] # Save cleaned data back to CSV  
df.to_csv('cleaned_data.csv', index=False)
```

Common Pandas Operations

`df.head()` — shows the first 5 rows.

`df.dropna()` — removes rows containing null values.

`df.fillna(0)` — replaces null values with 0.

`df.dtypes` — shows the data type of each column.

`df['col'] = pd.to_datetime(df['col'])` — converts a column to datetime format.

4. Parsing XML Data

XML (eXtensible Markup Language) is used by many older APIs and RSS feeds. Python's built-in **xml.etree.ElementTree** module handles XML parsing without any extra installs.

Example: Parsing an XML Response

```
import requests
import xml.etree.ElementTree as ET

url = 'https://example.com/feed.xml'
response = requests.get(url) # Parse the XML content
root = ET.fromstring(response.content) # Loop through all 'item' tags
for item in root.findall('.//item'):
    title = item.find('title').text
    link = item.find('link').text
    print(f'Title: {title}')
    print(f'Link: {link}')
```

Key XML Methods

`ET.fromstring(content)` — parses XML from a string.

`ET.parse('file.xml')` — parses XML from a local file.

`root.findall('.//tag')` — finds all matching tags anywhere in the tree.

`element.find('tag').text` — gets the text content of a child tag.

`element.attrib` — returns a dictionary of the element's attributes.

5. Working with REST APIs

REST APIs return structured data (usually JSON) in response to HTTP requests. Many APIs require an API key passed as a header or URL parameter for authentication.

Example: API Request with Key and Headers

```
import requests
API_KEY = 'your_api_key_here'
url = 'https://api.openweathermap.org/data/2.5/weather'
# Pass parameters in the URL
params = { 'q': 'Johannesburg', 'appid': API_KEY, 'units': 'metric' }
response = requests.get(url, params=params)
if response.status_code == 200:
    data = response.json()
    print('Temp:', data['main']['temp'])
else:
    print('Error:', response.status_code)
```

HTTP Status Codes to Know

- 200 — OK. Request was successful.
- 401 — Unauthorized. API key is missing or invalid.
- 403 — Forbidden. You don't have access to this resource.
- 404 — Not Found. The URL or endpoint doesn't exist.
- 429 — Too Many Requests. You've exceeded the rate limit.

6. Cleaning Scraped Data

Raw scraped data is rarely clean. Before storing or analyzing it you typically need to handle missing values, fix data types, remove duplicates, and standardize formats.

Full Cleaning Pipeline Example

```
import pandas as pd from datetime import datetime df = pd.read_csv('raw_data.csv') #  
1. Remove duplicate rows df = df.drop_duplicates() # 2. Remove rows with null values  
df = df.dropna() # 3. Convert date column to datetime df['date'] =  
pd.to_datetime(df['date']) # 4. Strip whitespace from string columns df['name'] =  
df['name'].str.strip() # 5. Convert temperature to float df['temperature'] =  
df['temperature'].astype(float) # 6. Filter out invalid values df =  
df[df['temperature'] > -50] # 7. Save cleaned data df.to_csv('clean_data.csv',  
index=False) print(f'Clean rows: {len(df)}')
```

Quick Reference

HTML — Library: requests + BeautifulSoup

Key methods: `soup.find()`, `find_all()`

JSON — Library: requests + json

Key methods: `response.json()`, `json.load()`

CSV — Library: pandas

Key methods: `pd.read_csv()`, `df.dropna()`

XML — Library: xml.etree.ElementTree

Key methods: `ET.fromstring()`, `findall()`

REST API — Library: requests

Key methods: `requests.get(url, params={})`

End of Tutorial